

Инкапсуляция, свойства, наследование и полиморфизм

Цель работы:

Научиться применять принципы ООП: скрытие данных через инкапсуляцию, использование свойств для контроля доступа, создание иерархий через наследование и реализацию полиморфного поведения.

ТЕОРИЯ

Инкапсуляция в Python

В Python нет строгих приватных полей, но есть соглашения:

`_name` — защищённый (protected) — не стоит использовать напрямую

`__name` — приватный (private) — Python изменяет имя (`_ClassName__name`)

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner          #
        self._type = "debit"        #
        self.__balance = balance    #
        # Публичный метод для доступа к
        # приватному полю
        def get_balance(self):
            return self.__balance
```

```
    def set_balance(self, value):
        if value >= 0:
            self.__balance = value
        else:
            raise ValueError("Баланс не
            может быть отрицательным")

acc = BankAccount("Иван", 1000)
print(acc.owner)          # ✓ Иван
print(acc._type)          # ✓ работает,
# print(acc.__balance)    # ×
# AttributeError
print(acc.get_balance())  # ✓ 1000
```

Свойства класса (property)

Свойства позволяют обращаться к методам как к полям.

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    # Геттер
    @property
    def celsius(self):
        return self._celsius

    # Сеттер
    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Ниже абсолютного нуля нельзя")
        self._celsius = value

    # Вычисляемое свойство (только чтение)
```

```
    @property
    def fahrenheit(self):
        return self._celsius * 9/5 + 32

temp = Temperature(25)
print(temp.celsius)      # 25 (как поле)
temp.celsius = 30        # использует
сеттер
print(temp.fahrenheit)   # 86.0
(вычисляется)
```

Виды свойств:

Тип	Реализация
Только чтение	только @property
Чтение/запись	@property + @имя.setter
Только запись	только @имя.setter (редко)
Вычисляемое	@property без поля

Наследование

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Издаёт звук"

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)  # ВЫЗОВ
конструктора родителя
        self.breed = breed

    def speak(self):            #
переопределение метода
        return "Гав!"

    def fetch(self):            # НОВЫЙ
метод
        return f"{self.name} приносит
мяч"

dog = Dog("Рекс", "Овчарка")
print(dog.name)    # Рекс (унаследовано)
print(dog.speak()) # Гав!
(переопределено)
print(dog.fetch()) # Рекс приносит мяч
(новое)
```

Полиморфизм

Способность объектов разных классов отвечать на одни и те же вызовы по-своему.

```
class Cat(Animal):
    def speak(self):
        return "Мяу!"

class Cow(Animal):
    def speak(self):
        return "Муу!"

def animal_sound(animal):
    """Полиморфная функция — работает с
    любым наследником Animal"""
    print(f"{animal.name} говорит:
    {animal.speak()}")

animals = [Dog("Бобик", "Дворняга"),
Cat("Мурка"), Cow("Бурёнка")]
for a in animals:
    animal_sound(a)

# Вывод:
# Бобик говорит: Гав!
# Мурка говорит: Мяу!
# Бурёнка говорит: Муу!
```

ПРАКТИЧЕСКОЕ ЗАДАНИЕ

Задача 1: Банковский счёт (инкапсуляция + свойства)

Создайте класс BankAccount:

Требования:

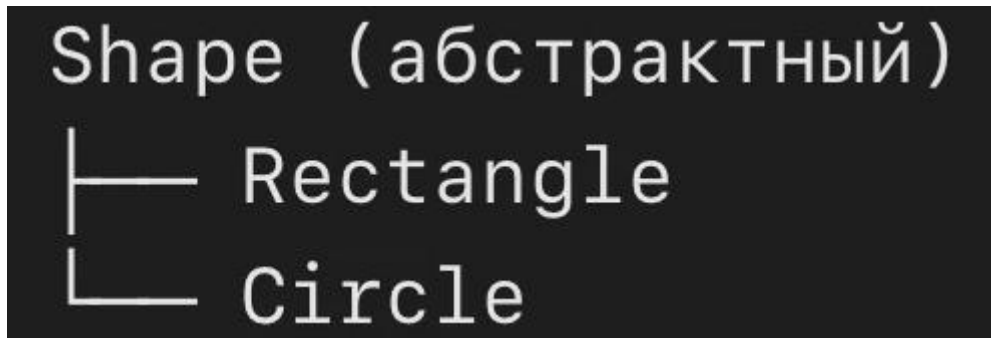
1. Приватные поля: `__account_number`, `__balance`
2. Публичное поле: `owner`
3. Свойство `balance` — только чтение (геттер)
4. Свойство `account_number` — только чтение
5. Свойство `is_negative` — вычисляемое (возвращает `True` если баланс < 0)
6. Методы: `deposit(amount)`, `withdraw(amount)` (с проверкой достаточности средств)

Код для проверки:

```
acc = BankAccount("Петр", 5000)
print(acc.owner)           # Петр
print(acc.balance)         # 5000
acc.deposit(1000)
print(acc.balance)         # 6000
acc.withdraw(2000)
print(acc.balance)         # 4000
print(acc.is_negative)     # False
# acc.__balance = 9999     # Должно
# выдать ошибку
```

Задача 2: Геометрические фигуры (наследование + полиморфизм)

Создайте иерархию по нижепредставленному тексту:



Базовый класс Shape:

- Абстрактные методы: `area()`, `perimeter()`

Класс Rectangle:

- Поля: `width`, `height` (свойства с проверкой >0)
- Реализует `area()` и `perimeter()`

Класс Circle:

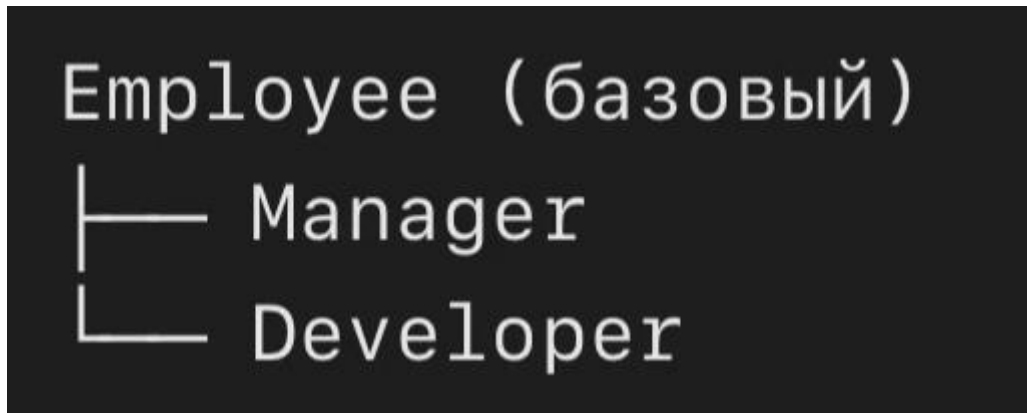
- Поле: `radius` (свойство >0)
- Реализует `area()` (πr^2) и `perimeter()` ($2\pi r$)

Полиморфная функция:

```
def print_shape_info(shape: Shape):  
    print(f"{type(shape).__name__}:  
    площадь = {shape.area():.2f}, периметр =  
    {shape.perimeter():.2f}")
```

Задача 3: Сотрудники компании (полный ООП)

Создайте иерархию:



Базовый класс Employee:

- **Приватные поля:** `__name`, `__base_salary`
- **Свойство** `salary` (геттер)
- **Абстрактный метод** `calculate_bonus()`

Класс Manager:

- **Дополнительное поле:** `team_size`
- **Бонус** = 10% от зарплаты * `team_size`

Класс Developer:

- **Дополнительное поле:** `programming_language`
- **Бонус** = 5% от зарплаты

Расширенное задание (Если захочется сделать):

Добавьте свойство `total_compensation` (зарплата + бонус)

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Чем отличается `_variable` от `__variable` в Python?
2. Что произойдёт, если попытаться получить доступ к `__variable` извне класса?
3. Для чего нужен декоратор `@property`?
4. В чём разница между переопределением (`override`) и перегрузкой (`overload`)? Почему в Python нет традиционной перегрузки методов?
5. Как вызвать метод родительского класса внутри класса-наследника?
6. Что такое "утиная типизация" в контексте полиморфизма Python?